

vound-remixed — Presentation Prep

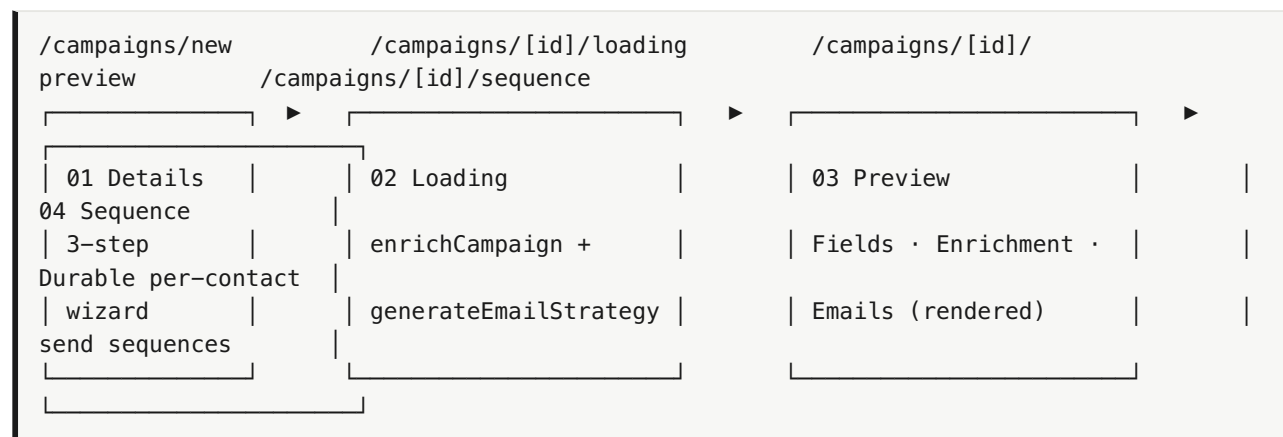
A single-document briefing you can read straight through before you present: what the app does, how it uses Workflow DevKit and the AI SDK, the code that actually makes it work, and the handful of narrative beats worth landing.

Live: <https://vound-remixed.vercel.app>

1. One-paragraph pitch

vound-remixed is an outbound GTM app. A user describes a campaign, drops in a CSV of contacts, and the system does the boring parts: it researches each company and person, drafts an email strategy, lets the user review everything in one workspace, and then runs a durable multi-step email sequence that pauses for days between sends and exits early if the prospect replies or bounces. The whole app is built on Vercel primitives — Fluid Compute, the AI Gateway, Workflow DevKit, and a Marketplace-provisioned Supabase — and the pipeline is designed so the LLM writes variables, not emails.

2. The four phases



Phase transitions are guarded server-side:

- `/loading` → redirects to `/preview` once `campaigns.email_strategy` is set.
- `/preview` → redirects to `/loading` if the strategy isn't there yet.
- `/details/[id]` → read-only snapshot. The wizard stays at `/campaigns/new`.

The phase tracker becomes fully free-navigable once `email_strategy` is populated — the Details page passes `unlockFuture={strategyReady}` so you can bounce forward from a post-submit snapshot.

3. Workflow DevKit — where and why

Every long-running operation is a Workflow. Four workflow entry points:

Workflow	File	Trigger	Why a workflow
<code>enrichCampaignWorkflow</code>	<code>workflows/ enrich- campaign.ts</code>	<code>defineEnrichmentAction</code> (after submit)	Fan-out across every account + contact with bounded concurrency. Takes minutes. Survives cold starts and redeploys. Emits SSE events the loading page subscribes to.
<code>generateEmailStrategyWorkflow</code>	<code>workflows/ generate- email- strategy.ts</code>	Chained from enrich's last step via <code>start()</code> inside a step	Runs the AI strategist + Zod validation + DB write. Separated so enrichment can finish even if the strategist fails (user can retry).
<code>launchCampaignWorkflow</code>	<code>workflows/ launch- campaign.ts</code>	"Create and Launch" button in Preview	Fans out one durable sequence workflow per contact. Returns immediately.
<code>sendContactSequenceWorkflow</code>	<code>workflows/ send- contact- sequence.ts</code>	Launched per-contact	Days-long durable loop. <code>Promise.race([sleep, hook])</code> drives step advancement and early exits.

Why Workflow DevKit instead of plain server actions or a queue

1. **Durability.** The sequence workflow lives for days waiting for the next step or a reply. A crashed instance picks up exactly where it left off. No cron, no polling, no external state machine.
2. **Sleep + hook as a single primitive.** `Promise.race([sleep("3d"), hook])` is the entire scheduling engine. The hook is resumed from the Resend webhook. No queue reconciliation.
3. **Step-level caching.** Every `"use step"` function's output is persisted. A retry doesn't re-issue AI calls or re-insert rows.
4. **Sandbox-free Node in steps.** Workflow functions run sandboxed; steps have full Node access. The pattern drives a clean separation — workflows orchestrate, steps do I/O.

5. **Streams double as APIs.** `getWritable<T>()` inside the workflow writes the default stream.

`run.getReadable()` from a route handler returns it. That's how the loading page's live event feed works — no extra transport.

Example 1 — The enrichment workflow entry point

Orchestration only. Every I/O call is a step.

```
// workflows/enrich-campaign.ts (line 349)
export async function enrichCampaignWorkflow(campaignId: string) {
  "use workflow";

  const { spec, accounts, contacts } = await loadCampaignData(campaignId);
  await emit({ type: "start", accounts: accounts.length, contacts: contacts.length });

  // Enrich accounts with bounded concurrency (CONCURRENCY = 3).
  await runBatched(accounts, CONCURRENCY, async (acc) => {
    await markStatus("accounts", acc.id, "enriching");
    await emit({ type: "account", id: acc.id, status: "enriching" });
    const result = await enrichAccountStep(acc.id, { ... }, spec);
    await emit({
      type: "account",
      id: result.id,
      status: result.ok ? "enriched" : "failed",
      error: result.error,
    });
  });

  // Re-read account context so contacts benefit from enriched company info.
  const accountContext = await fetchAccountContextMap(campaignId);

  await runBatched(contacts, CONCURRENCY, async (contact) => {
    // ...same shape: emit → enrichContactStep → emit
  });

  await finalizeCampaign(campaignId);
  await kickOffStrategy(campaignId); // starts generateEmailStrategyWorkflow
  await emit({ type: "done" });

  return { campaignId, accounts: accounts.length, contacts: contacts.length };
}
```

Example 2 — Streaming events to the browser

The workflow emits events via `getWritable<T>()`. The route handler returns the readable side directly. No SSE wrapper code; the Workflow stream is the payload.

```
// workflows/enrich-campaign.ts
async function emit(event: EnrichmentEvent) {
  "use step";
  const writer = getWritable<EnrichmentEvent>().getWriter();
  try {
    await writer.write(event);
  } finally {
    writer.releaseLock();
  }
}
```

```
// app/api/campaigns/[id]/enrich/route.ts
import { start, getRun } from "workflow/api";
import { enrichCampaignWorkflow } from "@workflows/enrich-campaign";

export async function POST(_req: Request, { params }: Params) {
  const { id } = await params;
  const run = await start(enrichCampaignWorkflow, [id]);
  return Response.json({ runId: run.runId });
}

export async function GET(req: Request, { params }: Params) {
  const url = new URL(req.url);
  const runId = url.searchParams.get("runId");
  const run = getRun(runId!);
  if (!(await run.exists)) {
    return Response.json({ error: "run not found" }, { status: 404 });
  }
  return new Response(run.getReadable(), {
    headers: {
      "Content-Type": "application/x-ndjson",
      "Cache-Control": "no-cache, no-transform",
    },
  });
}
```

Example 3 — Sleep + hook race for durable send sequences

The per-contact workflow's core scheduling loop. `sleep()` suspends the workflow; `createHook()` returns a resumable waitable; `Promise.race` picks whichever fires first.

```
// workflows/send-contact-sequence.ts (line 246)
export async function sendContactSequenceWorkflow(
  campaignId: string,
  contactId: string,
) {
  "use workflow";

  const ctx = await loadContext(campaignId, contactId);
  const { strategy, demoMode, renderCtx, already_sent_steps } = ctx;
  const dayUnitSeconds = demoMode ? DEMO_DAY_SECONDS : 86400;

  for (let i = 0; i < strategy.sequence.length; i++) {
    const step = strategy.sequence[i]!;
    if (already_sent_steps.includes(step.step)) continue;

    // Wait until this step's day offset, racing any reply/bounce on prev step.
    const prevOffset = i === 0 ? 0 : strategy.sequence[i - 1]!.day_offset;
    const delta = Math.max(0, step.day_offset - prevOffset);
    if (delta > 0) {
      const prevHook = createHook<HookPayload>({
        token: contactHookToken(contactId, strategy.sequence[Math.max(0, i - 1)]!.step),
      });
      const seconds = delta * dayUnitSeconds;

      const exitDuringWait = await Promise.race([
        (async () => { await sleep(`${seconds}s`); return { kind: "timeout" as const }; })(),
        (async () => { const evt = await prevHook; return { kind: "event" as const, evt }; })(),
      ]);

      if (exitDuringWait.kind === "event") {
        await handleTerminalEvent(contactId, i - 1, exitDuringWait.evt);
        return { campaignId, contactId, exited: exitDuringWait.evt.kind };
      }
    }

    const rendered = renderStep(strategy, step, renderCtx);
    const sendId = await persistQueuedSend(campaignId, contactId, step.step,
      rendered.subject, rendered.body);
    await sendEmailStep(sendId, renderCtx.contact.email, rendered.subject,
      rendered.body, demoMode);
    // ... last-step grace window: another sleep/hook race ...
  }
}
```

Example 4 — Resuming the hook from a webhook

Resend fires a webhook; we route it straight into `resumeHook()`. The workflow's `Promise.race` wakes up and branches on the payload.

```
// app/api/webhooks/resend/route.ts
import { resumeHook } from "workflow/api";
import { contactHookToken, type HookPayload } from "@lib/sequence/types";

// ... after matching the send row for the webhook event ...
await resumeHook(
  contactHookToken(sendRow.contact_id, sendRow.step),
  { kind } satisfies HookPayload,
);
```

The hook token scheme is deliberately deterministic so the webhook handler doesn't need to know which run to target — just "which contact, which step."

```
// lib/sequence/types.ts
export function contactHookToken(contactId: string, step: number): string {
  return `seq:${contactId}:${step}`;
}

export type HookPayload =
  | { kind: "reply" }
  | { kind: "bounce" }
  | { kind: "complaint" }
  | { kind: "delivered" }
  | { kind: "opened" };
```

Talking points about Workflow in this app

- **Demo mode compresses days into seconds.** `DEMO_DAY_SECONDS = 30` — a "3-day" wait becomes 90 seconds on stage while keeping the same code path.
- **Step 1 of the strategy has `day_offset: 0`** — the first send is immediate, the waits come between subsequent steps.
- **Workflows self-chain.** `enrichCampaignWorkflow` ends by calling `start(generateEmailStrategyWorkflow, ...)` inside a step, so the user arrives at `/preview` with the strategy already running.

4. AI SDK — where, when, and why

All AI calls route through the **Vercel AI Gateway** using the plain `"anthropic/claude-sonnet-4.6"` string model id. Auth is OIDC (`VERCEL_OIDC_TOKEN`) — no Anthropic key in the project. Every call uses `generateText({ output: Output.object({ schema }) })` with a Zod schema, so the model returns a parsed, typed object. No JSON regex, no retry-on-parse scaffolding.

The four calls

#	Call	File	Model inputs	Model output	When
1	Enrichment-spec strategist	<code>lib/ai/generate-enrichment-spec.ts</code>	Campaign brief, ranked goals, detected CSV columns	<code>EnrichmentSpec</code> — up to 5 custom account fields + 5 contact fields, each with a source + query template, plus personalization hooks and qualification rubrics	Server action, right after wizard submit, before the row is inserted
2	Per-row enrichment (account)	<code>lib/enrichment/ai-enrichment.ts</code> (<code>aiEnrichAccount</code>)	Standard account fields + the enrichment spec's custom-field catalog	Filled standard fields + <code>custom_data</code> per the spec	Inside the enrichment workflow, once per account, reconciled against a provider-lookup stub
3	Per-row enrichment (contact)	<code>lib/enrichment/ai-enrichment.ts</code> (<code>aiEnrichContact</code>)	Standard contact fields + account context (for correlation) + contact custom-field catalog	Filled fields + <code>custom_data</code>	Inside the enrichment workflow, once per contact
4	Email strategist	<code>lib/ai/generate-email-strategy.ts</code>	Enrichment spec + campaign brief	<code>EmailStrategy</code> — framework choice (PVP/PQS/Eric), token list, personalization rules, sequence of <code>EmailStep</code> templates with subject variants + body templates	Inside <code>generateEmailStrategyWorkflow</code> , after enrichment finishes

#	Call	File	Model inputs	Model output	When
5	Lazy company-summary	lib/enrichment/generate-company-summary.ts	One account's enriched row	{ summary, angle, insights }	On first open of a company insight card in Preview, persisted to accounts.summary/angle/insights

Canonical call shape

Every call looks like this. The `Output.object({ schema })` is the point — it turns the LLM into a function that returns typed, Zod-validated data.

```
// lib/ai/generate-enrichment-spec.ts
import { generateText, Output } from "ai";
import { enrichmentSpecSchema } from "@lib/schema/enrichment-spec";

const MODEL_ID = "anthropic/claude-sonnet-4.6"; // routed via AI Gateway

const { output } = await generateText({
  model: MODEL_ID,
  system: SYSTEM_PROMPT,
  prompt: buildUserPrompt(input),
  output: Output.object({ schema: enrichmentSpecSchema }),
  maxOutputTokens: 16000,
});
return output; // already typed as EnrichmentSpec
```

Same shape in all four modules. Only the system prompt and schema change.

Example — the strategist's output rules, in the prompt

The strategist picks between three frameworks and emits a token/rule DSL. The system prompt teaches the model the DSL, then the response schema enforces it.

```
## Token and rule DSL

Tokens are referenced in templates as:
- {{first_name}}, {{last_name}}, {{email}}, {{job_title}}, ...
- {{company.name}}, {{company.domain}}, {{company.description}}, ...
- {{custom.<key>}}          - contact-level custom field
- {{account.custom.<key>}} - account-level custom field
```

And the framework brief, lifted verbatim from the prompt:

PVP (Permissionless Value Prop) — max 3 emails. Use when the enriched data has high signal density. The value proposition is the research itself.

PQS (Pain-Qualified Segment) — max 4 emails. Use when the campaign targets a named pain in a tight segment.

Eric (Firmographic) — max 4 emails. Use for firmographic data. E1 net new → E2 thread bump → E3 net new → E4 thread bump.

The deterministic renderer (`lib/email/render-template.ts`) consumes the strategy: two-pass substitution (expand `{{token.x}}` → resolve `{{custom.y}}` paths) and a forgiving `company.*` path resolver that accepts both `company.name` and `company.company_name` .

The design principle

The LLM writes variables, not emails.

The strategist designs templates with clear conditional slots. The runtime substitutes. Per-contact output is deterministic, auditable, and immune to model drift on individual sends. A bug in a rule or a typo in a template is fixable in one place; the fix affects every contact on next render.

Why the AI Gateway matters here

- **Zero key-management.** OIDC means no provider keys anywhere in the project — auth is via `VERCEL_OIDC_TOKEN`. `vercel env pull` wires up a short-lived token; production auto-refreshes.
- **One string changes the model.** The whole pipeline uses a plain string id. Swapping from Anthropic to OpenAI or Gemini is a one-line change.
- **Unified observability.** Every call is tagged and visible in the Gateway dashboard with token counts and routing info.
- **Built-in failover.** Not yet wired, but adding `providerOptions.gateway` with an `order` list would give automatic cross-provider failover.

5. Project structure

```

vound-remixed/
├── app/
│   ├── (dashboard)/layout.tsx      Next.js 16 App Router
│   ├── (dashboard)/campaigns/
│   │   ├── new/
│   │   │   ├── page.tsx           CampaignForm shell
│   │   │   └── actions.ts         defineEnrichmentAction: AI spec gen +
│   │   │                                   DB inserts + starts enrich workflow
│   │   └── [id]/
│   │       ├── page.tsx           Redirects to correct phase based on state
│   │       ├── details/page.tsx   Read-only brief snapshot
│   │       ├── loading/page.tsx   Phase 2 shell
│   │       ├── preview/
│   │       │   ├── page.tsx
│   │       │   └── actions.ts     updateEmailStrategyAction +
│   │       │                                   launchCampaignAction
│   │       └── sequence/
│   │           ├── page.tsx
│   │           └── actions.ts     Phase 4: per-contact send state
│   └── api/
│       ├── campaigns/[id]/enrich/route.ts    POST=start, GET=stream
│       ├── campaigns/[id]/messaging/route.ts Same shape for strategy run
│       ├── campaigns/[id]/status/route.ts     JSON health check (polling)
│       ├── campaigns/[id]/accounts/[accountId]/summary/route.ts
│       │                                   Lazy per-company summary
│       ├── campaigns/[id]/sequence/route.ts   Launch
│       ├── campaigns/[id]/sequence/simulate/route.ts Demo-mode send events
│       └── webhooks/resend/route.ts            Resend → resumeHook
├── workflows/
│   └── "use workflow" entry points
│       ├── enrich-campaign.ts           Phase 2a
│       ├── generate-email-strategy.ts   Phase 2b (chained)
│       ├── launch-campaign.ts           Phase 4 boot
│       └── send-contact-sequence.ts      Durable per-contact loop
├── lib/
│   ├── ai/
│   │   ├── generate-enrichment-spec.ts    Call 1 – strategist
│   │   └── generate-email-strategy.ts     Call 4 – email strategist
│   ├── enrichment/
│   │   ├── ai-enrichment.ts              Calls 2-3 – per-row enrichment
│   │   ├── generate-company-summary.ts    Call 5 – lazy summary
│   │   ├── mock-provider.ts              Placeholder for real CRM/data lookups
│   │   └── reconcile.ts                   Merge provider hits with AI output
│   ├── email/
│   │   ├── render-template.ts             Token/rule substitution (2-pass)
│   │   └── resend.ts                      Resend client + demo-mode stub
│   ├── sequence/
│   │   ├── types.ts                       HookPayload, contactHookToken()
│   │   └── state.ts                       Allowed state transitions
│   ├── schema/
│   │   └── campaign-input.ts              Wizard form schema

```

└─ enrichment-spec.ts	Strategist output schema
└─ email-strategy.ts	Email strategy output schema
└─ supabase/	
└─ server.ts	Admin client (service role)
└─ client.ts	Browser SSR client
└─ csv/parse.ts	Papaparse + header normalization
└─ loading/witty-messages.ts	Phase 2 copy rotation
└─ phases.ts	PhaseId + phaseHref()
└─ campaign-goals.ts	Static goal catalog
└─ components/	
└─ campaign-form/	Wizard shell, steps, providers grid
└─ phase-tracker/	Top-of-page phase nav
└─ loading/	SSE-subscribed loading screen
└─ preview/	Tabs, tables, cell popovers, company insights, emails tab
└─ sequence/	Per-contact state board
└─ supabase/migrations/	Version-controlled schema (5 migrations)
└─ scripts/	node --env-file seeding scripts
└─ sample-data/	Test CSVs
└─ AGENTS.md	Heads-up that Next.js 16 has breaking changes vs. training data

6. Database schema, in one paragraph

Six tables: `campaigns` (the brief + state + generated spec + strategy), `accounts` (companies, one per unique corporate domain, with lazy summary fields), `contacts` (people, FK to accounts, enriched fields plus a `raw_import` jsonb of CSV extras), `email_sends` (one row per scheduled + sent email, driven by the send workflow), `sequence_events` (audit trail), `provenance` (per-field source tracking). Campaigns store both `enrichment_run_id` and `strategy_run_id` so the UI can reattach to the right Workflow stream on refresh.

7. Things to have ready for Q&A

"What happens if the AI times out?"

AI SDK calls throw; steps retry with backoff automatically (Workflow DevKit default). `FatalError` in the strategist kills the enrichment workflow with a visible failure state. A "Restart strategist" button POSTs to the messaging route which `start()` s a fresh workflow run.

"What if someone refreshes during loading?"

The campaign row stores `enrichment_run_id` / `strategy_run_id`. On refresh, the loading page queries `/api/campaigns/[id]/status`, gets the current run ids, and re-subscribes to the existing stream via `GET /enrich?runId=...` or `GET /messaging?runId=...`. The user sees replayed events, then live ones.

"Why a custom render-template instead of letting the AI write each email?"

Three reasons. (a) Cost — one strategy call serves every contact. (b) Determinism — a typo in a template is a single fix, not a per-contact retry. (c) Auditability — every rendered email can trace back to a specific rule + token set. The rendering pipeline includes a tolerant `company.*` resolver (both `company.name` and `company.company_name` work) and a two-pass substitution so tokens that expand to `{{custom.x}}` placeholders get re-resolved in the second pass.

"How is the per-contact sending loop actually durable?"

The workflow is *literally suspended* during `sleep("3d")` — it's not a running process waiting. State is persisted. A hook resumes it by token. If we redeploy the app mid-wait, the workflow wakes up on the new version and continues. This is what makes Workflow DevKit different from a `setTimeout-on-a-serverless-function` approach: the platform owns the durability.

"What's the Vercel piece of this?"

- **Fluid Compute** runs Next.js routes + server actions. Middleware has full Node, no edge restriction.
- **AI Gateway** routes every model call with OIDC auth.
- **Workflow DevKit** is the durable runtime. Observable in the Vercel Dashboard — `npx workflow web <runId>` opens the run viewer.
- **Supabase (Marketplace)** is the database. Envs provisioned by `vercel env pull`.
- **Deploys** happen via `vercel --prod` from the working tree, or auto from `git push` on `main`. We've used the CLI path for this project.

"What did you simplify vs. the original vound?"

The original was a six-phase wizard: Input → Fields → Data → Messaging → Sequence → Report. Users got lost in the middle — they submitted a brief and then had to review four intermediate artifacts before sending anything. vound-remixed collapses Fields + Data + Messaging into one Preview tab and deletes the Report phase entirely. The narrative is now "**brief** → **wait** → **review** → **launch**", which is how users actually think about it.

8. Demo cheat sheet

1. Open <https://vound-remixed.vercel.app>.
 2. Click into the seeded Austin campaign (Hangar Bar follow-up) if present, or create a new one: pick "Post-event follow-up", drop the Hangar Bar pitch, upload a CSV, pick a few sources, submit.
 3. Phase 2 shows witty-message rotation plus live event ticker. Call out that each tick is a `getWritable()` event from the workflow.
 4. Land on Preview. Show Fields tab (the enrichment spec), Enrichment tab (the actual researched data — double-click any cell to show the provenance popover), Emails tab (the strategy, rendered for one contact).
 5. Click into a company card → lazy summary generates (this is a 5th AI call, fired on-demand, persisted to `accounts.summary`).
 6. Back on Emails, pick "Create and Launch" — this fires `launchCampaignWorkflow`, which fans out one sequence workflow per contact.
 7. In a separate tab, `npx workflow web <runId>` shows the live runs.
-

9. Final nuance that lands well

- **Every minute of work in this app is a Workflow, not a request.** That's the story. Enrichment is a workflow. Strategy generation is a workflow. Every contact's multi-day send journey is a workflow. None of it is tied to an HTTP request lifecycle.
- **Every AI call is a schema-enforced function, not a text endpoint.** Zod + `Output.object` means the model either returns valid data or throws. No regex. No "extract JSON from fences."
- **The UI is a thin subscriber to those workflows.** SSE streams from Workflow runs, redirects guarded on DB state, polling fallbacks for dropped streams. The UI never drives state — it just reflects it.